AI Knowledge Assistant

RAG Platform — Enterprise Project Plan

ASP.NET Core 8 · Semantic Kernel · Ollama · Qdrant · React · PostgreSQL

Clean Architecture · CQRS · MediatR · JWT · Docker · 100% Free & Open Source

Prepared for: Abhishek | Duration: 4 Weeks | Portfolio: GitHub / LinkedIn

4 Weeks	100%	8+ Services	Full-Stack	Offline AI
Timeline	Free Tools	Microservices	React + .NET 8	Ollama LLM

TABLE OF CONTENTS

1. PROJECT INTRODUCTION & VISION

What Is This Project?

The AI Knowledge Assistant is a production-grade, enterprise-class Retrieval-Augmented Generation (RAG) platform built entirely on free and open-source tools. It allows users to upload documents (PDFs, Word files, text), which are automatically chunked, embedded into a vector database, and made queryable through a natural-language chat interface powered by a locally-running Large Language Model (LLM) via Ollama — with zero API cost and zero data leaving your machine.

Unlike cloud-based AI assistants (OpenAI, Azure OpenAI), this system runs entirely offline. Businesses in healthcare, legal, finance, and defence — domains with strict data privacy requirements — can use this to build internal knowledge bases without any compliance risk.

Why This Project for Your Portfolio?

This project hits every evaluation criterion a Senior SDE interviewer at Mastercard, JPMorgan, Microsoft, or SAP checks: Clean Architecture, CQRS, event-driven design, containerisation, AI/ML integration, security (JWT), and modern frontend. It is rare, differentiated, and directly maps to the GenAI + platform engineering wave every enterprise is investing in right now (2025-2026).

Core Value Proposition

- **Fully Offline:** Llama 3 / Mistral running via Ollama — no OpenAI key needed, zero cloud cost.
- **Enterprise Patterns:** Clean Architecture, CQRS/MediatR, repository pattern, domain events.
- **Production-Ready:** Docker Compose, health checks, structured logging (Serilog), JWT auth.
- **Full-Stack Showcase:** React frontend with real-time streaming chat responses via Server-Sent Events.
- **Demonstrable AI Knowledge:** RAG pipeline, vector embeddings, semantic search — in-demand skills.

2. SCOPE & BOUNDARIES

✓ IN SCOPE	✗ OUT OF SCOPE
• Document ingestion: PDF, DOCX, TXT upload and processing	• Multi-tenant SaaS billing (single-user/team scope only)
• Text chunking with configurable chunk size and overlap	• Fine-tuning or training custom LLM models
• Embedding generation using Ollama (nomic-embed-text model)	• Production Kubernetes deployment (Docker Compose only)
• Vector storage and similarity search using Qdrant	• Mobile app (web responsive only)
• Contextual response generation using Llama 3 / Mistral via Ollama	• Real-time multi-user collaboration
• Conversation history management (per-session chat threads)	• Enterprise SSO / Active Directory integration
• JWT-based user authentication and authorisation	
• React frontend with chat UI, document manager, and upload panel	
• RESTful API with Swagger/OpenAPI documentation	
• Docker Compose deployment (all services containerised)	
• PostgreSQL for user/document metadata persistence	
• GitHub repository with proper README and architecture diagrams	

3. REAL-WORLD USAGE & BUSINESS VALUE

RAG platforms are the #1 enterprise AI use case in 2025-2026. Every major corporation is building internal knowledge assistants to help employees query internal documentation, legal agreements, HR policies, and technical runbooks without sending sensitive data to the cloud.

Healthcare / Siemens Healthineers

Query MRI scanner manuals, clinical protocols, regulatory compliance documents, and FDA/CE submissions — all offline, HIPAA/GDPR compliant. Direct alignment with your current employer's domain.

Legal & Compliance (JPMorgan, Goldman Sachs)

Lawyers query thousands of case files, contracts, and regulatory circulars. RAG enables sub-second semantic search across millions of documents without sending client data to OpenAI.

IT Operations & DevOps

Chat with runbooks, incident post-mortems, architecture docs, and Confluence wikis. Reduces mean time to resolve (MTTR) by giving on-call engineers instant contextual answers.

HR & Onboarding

New joiners ask natural language questions about company policies, benefits, and procedures. Reduces HR ticket volume by 40-60% in real enterprise deployments.

Product & Engineering Knowledge Base

Engineers chat with API docs, design decision records (ADRs), and sprint retrospectives. Bridges the knowledge gap between senior engineers and new hires.

4. COMPLETE TECHNOLOGY STACK

BACKEND	
.NET 8 / ASP.NET Core 8	Web API framework — minimal APIs + controllers
C# 12	Primary programming language
Semantic Kernel (Microsoft)	Orchestration framework for LLM, memory, and skills
MediatR 12	CQRS mediator — decouples commands/queries from handlers
FluentValidation	Pipeline validation behaviours
Serilog + Seq	Structured logging with a free local log viewer UI
AutoMapper	Object-to-object mapping
Entity Framework Core 8	ORM for PostgreSQL (metadata/users only)
Dapper (optional)	Micro-ORM for read-side performance queries

AI / ML	
Ollama	Local LLM inference server — runs Llama 3, Mistral, Phi-3
nomic-embed-text (via Ollama)	Free text embedding model for vector generation
Llama 3 8B / Mistral 7B	Core LLM for answer generation (runs on CPU/GPU locally)
Semantic Kernel Memory	Abstraction over vector DB for RAG operations

VECTOR DATABASE	
Qdrant	High-performance vector database — Docker image, free & open source

RELATIONAL DATABASE	
PostgreSQL 16	User data, document metadata, conversation history
pgAdmin 4	Free GUI client for PostgreSQL

FRONTEND	
React 18	SPA framework with functional components and hooks
TypeScript	Type-safe frontend development
Tailwind CSS	Utility-first CSS framework

React Query (TanStack)	Server state management and caching
Zustand	Lightweight client state management
React Markdown	Render LLM markdown responses in chat
Axios	HTTP client for API calls

DEVOPS & TOOLING	
Docker Desktop (free)	Containerisation for all services
Docker Compose	Multi-service orchestration: API, Qdrant, Postgres, Ollama, React
GitHub	Version control and portfolio showcase
GitHub Actions (free tier)	CI pipeline — build + test on push
Visual Studio Code	Primary IDE (VS Code 2022 = VS Code with .NET 8 SDK)
Swagger / Scalar	API documentation — auto-generated
REST Client (VS Code ext.)	API testing from .http files in repo

SECURITY	
JWT Bearer Tokens	Stateless authentication
BCrypt.Net	Password hashing
CORS policy	Locked to React dev server and production origin
HTTPS (dev cert)	dotnet dev-certs https --trust

5. ARCHITECTURE OVERVIEW

Clean Architecture Layers

Domain Layer	Entities: Document, Chunk, Conversation, Message, User Domain Events: DocumentIngested, ChunkEmbedded Value Objects: ChunkId, EmbeddingVector Repository Interfaces (no implementation here)
Application Layer	CQRS Commands: IngestDocumentCommand, SendMessageCommand CQRS Queries: GetConversationsQuery, SearchChunksQuery MediatR Handlers, Pipeline Behaviours (validation, logging) DTO/ViewModel mapping, IUnitOfWork interface
Infrastructure Layer	EF Core + PostgreSQL (UserRepository, DocumentRepository) Qdrant adapter (IVectorRepository implementation) Ollama adapter (IEmbeddingService, ILlmService implementation) Semantic Kernel registration, JWT token service
Presentation Layer (API)	ASP.NET Core Minimal APIs / Controllers Swagger / Scalar UI JWT middleware, CORS, rate limiting Server-Sent Events endpoint for streaming chat

RAG Pipeline Flow

Step 1	**INGEST**	User uploads PDF/DOCX/TXT via React UI → POST /api/documents
Step 2	**EXTRACT**	Document text extracted using PdfPig / DocNet
Step 3	**CHUNK**	Text split into 512-token chunks with 50-token overlap (RecursiveTextSplitter)
Step 4	**EMBED**	Each chunk sent to Ollama (nomic-embed-text) → 768-dim vector returned
Step 5	**STORE**	Vectors + metadata stored in Qdrant; doc metadata in PostgreSQL
Step 6	**QUERY**	User types question in React chat → POST /api/chat/message
Step 7	**SEARCH**	Question embedded → Qdrant cosine similarity search → top-k chunks retrieved
Step 8	**AUGMENT**	Retrieved chunks + conversation history assembled into LLM prompt

Step 9	**GENERATE**	Llama 3 via Ollama streams answer → SSE to React frontend
Step 10	**DISPLAY**	React renders markdown response in real time, stores to conversation history

6. DEVELOPMENT ENVIRONMENT SETUP

Step 1 — Install Prerequisites

Download & Install .NET 8 SDK	`https://dotnet.microsoft.com/download/dotnet/8.0 (free)`
Install Node.js LTS (for React)	`https://nodejs.org (v20 LTS recommended)`
Install Docker Desktop	`https://www.docker.com/products/docker-desktop (free personal)`
Install Ollama	`https://ollama.ai — one-click installer for Windows`
Install VS Code	`https://code.visualstudio.com (note: not Visual Studio; VS Code is the free IDE)`
Install Git	`https://git-scm.com`

Step 2 — VS Code Extensions to Install

C# Dev Kit	`ms-dotnettools.csdevkit — IntelliSense, debugging for .NET`
Docker	`ms-azuretools.vscode-docker — Docker Compose GUI inside VS Code`
REST Client	`humao.rest-client — test .http files directly`
Prettier	`esbenp.prettier-vscode — frontend formatting`
GitLens	`eamodio.gitlens — enhanced Git history`
Thunder Client	`rangav.vscode-thunder-client — lightweight Postman alternative`

Step 3 — Pull Ollama Models (run in terminal)

Pull embedding model	`ollama pull nomic-embed-text`
Pull LLM (pick one)	`ollama pull llama3 OR ollama pull mistral`
Verify Ollama is running	`curl http://localhost:11434/api/tags`

Step 4 — Start Infrastructure via Docker Compose

Qdrant vector DB	`docker compose up qdrant -d → UI at http://localhost:6333/dashboard`
PostgreSQL	`docker compose up postgres -d → connect with pgAdmin`
Seq (log viewer)	`docker compose up seq -d → UI at http://localhost:5341`

Step 5 — Clone and Run the Project

Clone repo	`git clone https://github.com/YOUR_USERNAME/ai-knowledge-assistant`
Restore .NET packages	`cd src/API && dotnet restore`
Apply EF Core migrations	`dotnet ef database update`
Run API	`dotnet run` → `https://localhost:7001/swagger`
Run React frontend	`cd frontend && npm install && npm run dev` → `http://localhost:5173`

7. GITHUB REPOSITORY STRUCTURE

Recommended Folder Structure

```
ai-knowledge-assistant/ ■■■ src/ ■ ■■■ Domain/ # Entities, domain events, interfaces ■ ■■■ Application/ # CQRS handlers, validators, DTOs ■ ■■■ Infrastructure/ # EF Core, Qdrant adapter, Ollama adapter ■ ■■■ API/ # ASP.NET Core project, controllers, middleware ■■■ frontend/ # React + TypeScript + Tailwind ■ ■■■ src/ ■ ■ ■■■ components/ # Chat, DocumentList, UploadPanel ■ ■ ■■■ pages/ # Home, Login, Dashboard ■ ■ ■■■ hooks/ # useChat, useDocuments ■ ■ ■■■ api/ # Axios client, API types ■ ■■■ package.json ■■■ docker/ # Individual Dockerfiles ■■■ docker-compose.yml # Full stack orchestration ■■■ docker-compose.dev.yml # Dev overrides (hot reload) ■■■ .github/workflows/ci.yml # GitHub Actions CI ■■■ docs/ ■ ■■■ architecture.png # Architecture diagram (draw.io, free) ■ ■■■ rag-pipeline.png # RAG flow diagram ■ ■■■ demo.gif # Screen recording of app ■■■ .http/ # REST Client test files ■ ■■■ auth.http ■ ■■■ documents.http ■ ■■■ chat.http ■■■ README.md # Detailed README with badges
```

GitHub Best Practices: Use semantic commits (feat:, fix:, docs:). Add a detailed README with architecture PNG, tech badges (shields.io), GIF demo, and 'How to Run' section. Pin the repo on your GitHub profile. Add topics: dotnet, rag, semantic-kernel, ollama, qdrant, clean-architecture.

8 – 11. DETAILED 4-WEEK DEVELOPMENT PLAN

Daily Commitment: 2-3 hours/day on weekdays, 4-5 hours/day on weekends = ~70 hours total. Each week ends with a working, committable milestone. All tools are free.

WEEK 1 — FOUNDATION, PROJECT SETUP & BACKEND SKELETON

Goal: A running ASP.NET Core 8 solution with Clean Architecture scaffolded, Docker Compose up, JWT auth working, and document entity created in PostgreSQL.

WEEK 1	Foundation & Backend Skeleton
Day 1 (Mon)	Create GitHub repo. Scaffold solution: dotnet new sln, add Domain/Application/Infrastructure/API projects. Configure project references per Clean Architecture rules.
Day 2 (Tue)	Define Domain entities: Document, Chunk, Conversation, Message, User, ApplicationUser. Add domain events skeleton. Create repository interfaces in Domain.
Day 3 (Wed)	Set up EF Core 8 with PostgreSQL in Infrastructure. Create DbContext (AppDbContext). Write initial migration. Test connection via pgAdmin.
Day 4 (Thu)	Implement JWT auth: register/login endpoints, JwtTokenService, password hashing with BCrypt. Test with REST Client (.http file).
Day 5 (Fri)	Set up MediatR + FluentValidation pipeline. Write first command: RegisterUserCommand + handler. Add Serilog structured logging + Seq sink.
Weekend (Sat-Sun)	Write IngestDocumentCommand skeleton. Set up docker-compose.yml (postgres, qdrant, seq, api). Add Swagger/Scalar UI. Write README skeleton. Commit Week 1 milestone.

Deliverables: ✓ Solution structure committed ✓ JWT auth working ✓ Docker Compose up ✓ PostgreSQL migrations applied ✓ Swagger UI accessible

WEEK 2 — RAG PIPELINE: INGESTION, EMBEDDING & VECTOR SEARCH

Goal: Complete end-to-end document ingestion → chunking → embedding → Qdrant storage, plus semantic search returning relevant chunks for a query.

WEEK 2	RAG Pipeline — Ingestion & Vector Search
Day 8 (Mon)	Integrate PdfPig (free NuGet) for PDF text extraction. Implement IDocumentExtractor interface + PdfExtractor, TxtExtractor. Handle DOCX with DocumentFormat.OpenXml (free).
Day 9 (Tue)	Build TextChunkingService: recursive character splitter with configurable chunk size (512 tokens) and overlap (50 tokens). Unit test with sample documents.
Day 10 (Wed)	Implement OllamaEmbeddingService: call Ollama REST API (/api/embeddings) with nomic-embed-text. Handle batch embedding for all chunks of a document.
Day 11 (Thu)	Implement QdrantVectorRepository: IVectorRepository using Qdrant .NET SDK. Methods: UpsertVectorsAsync, SearchSimilarAsync (cosine similarity, top-k). Store chunk text as payload.
Day 12 (Fri)	Wire IngestDocumentCommand handler: extract → chunk → embed → store in Qdrant + PostgreSQL (metadata). Add progress tracking. Test full ingestion with a real PDF.
Weekend (Sat-Sun)	Implement SearchChunksQuery handler. Write SearchDocumentEndpoint. Test semantic search via .http file — verify relevant chunks returned. Commit Week 2 milestone.

Deliverables: ✓ PDF/DOCX ingestion working ✓ Embeddings stored in Qdrant ✓ Semantic search endpoint tested ✓ Week 2 branch merged to main

WEEK 3 — CHAT API, LLM INTEGRATION & REACT FRONTEND

Goal: Working chat API with Semantic Kernel + Llama 3/Mistral generating contextual answers; React frontend with chat UI, document manager, and auth pages.

WEEK 3	Chat API, LLM & React Frontend
Day 15 (Mon)	Register Semantic Kernel in DI container. Implement OllamaLlmService: ITextGenerationService wrapping Ollama API. Configure Semantic Kernel memory connector pointing to Qdrant.
Day 16 (Tue)	Build RAG prompt template: system prompt + retrieved context chunks + conversation history + user question. Implement SendMessageCommand handler using SK.
Day 17 (Wed)	Add Server-Sent Events (SSE) streaming endpoint: GET /api/chat/{id}/stream — stream Ollama token-by-token response to client. Test with curl.
Day 18 (Thu)	React project setup: Vite + TypeScript + Tailwind. Implement auth pages (Login, Register) with JWT storage in memory (not localStorage). Setup Axios interceptor for Bearer token.
Day 19 (Fri)	Build Chat UI component: message bubbles, markdown rendering (react-markdown), SSE hook (useEventSource), loading skeleton. Build DocumentList and UploadPanel components.
Weekend (Sat-Sun)	Wire React to API: upload document, start conversation, send message, receive streamed response. Add React Query for data fetching. Commit Week 3 milestone — record demo GIF.

Deliverables: ✓ Full RAG chat working end-to-end ✓ SSE streaming responses ✓ React UI functional ✓ Demo GIF recorded

WEEK 4 — POLISH, DOCKER, TESTING & PORTFOLIO LAUNCH

Goal: Production-quality Docker Compose setup, unit/integration tests, architecture diagrams, polished README, and LinkedIn post published.

WEEK 4	Polish, Docker, Testing & Portfolio Launch
Day 22 (Mon)	Write unit tests (xUnit + Moq) for: TextChunkingService, IngestDocumentCommandHandler (mock Qdrant + Ollama), SendMessageCommandHandler. Target 70%+ coverage on application layer.
Day 23 (Tue)	Write integration test: full ingestion → search → chat flow using Testcontainers (Qdrant + Postgres in Docker, free). Add WebApplicationFactory test for API endpoints.
Day 24 (Wed)	Dockerise API: multi-stage Dockerfile (build stage + runtime stage). Dockerise React: nginx serving production build. Add healthchecks to docker-compose.yml for all services.
Day 25 (Thu)	Set up GitHub Actions CI: on push to main — dotnet build, dotnet test, docker compose build. Add README badges: build status, .NET 8, Docker, license.
Day 26 (Fri)	Create architecture diagram with draw.io (free, browser-based). Create RAG pipeline flow diagram. Export as PNG and add to /docs folder and README.
Weekend (Sat-Sun)	Final polish: error handling, loading states in React, empty states. Write full README (500+ words). Write LinkedIn post with 3-bullet summary + demo GIF. Publish repo. Update resume.

Deliverables: ✓ Docker Compose full stack working ✓ CI pipeline green ✓ Tests passing ✓ README polished ✓ LinkedIn post published ✓ Resume updated

12. KEY API ENDPOINTS REFERENCE

Method	Endpoint	Description	Auth
POST	`/api/auth/register`	Register new user (name, email, password)	Public
POST	`/api/auth/login`	Login → returns JWT access token	Public
POST	`/api/documents`	Upload document (multipart/form-data)	Auth
GET	`/api/documents`	List all documents for current user	Auth
GET	`/api/documents/{id}/status`	Get ingestion status (processing/completed/failed)	Auth
DELETE	`/api/documents/{id}`	Delete document + Qdrant vectors	Auth
POST	`/api/conversations`	Create new conversation thread	Auth
GET	`/api/conversations`	List all conversations	Auth
POST	`/api/conversations/{id}/messages`	Send a message → returns full RAG response	Auth
GET	`/api/conversations/{id}/stream`	SSE stream — token-by-token LLM response	Auth
GET	`/api/conversations/{id}/messages`	Get message history for a conversation	Auth
GET	`/api/health`	Health check endpoint for Docker/k8s	Public

13. PORTFOLIO & LINKEDIN PRESENTATION TIPS

README Must-Haves

- Top section: project description (2-3 lines), shields.io badges (.NET 8, Docker, License, Build status)
- Architecture diagram (PNG from draw.io) above the fold
- Animated GIF demo (use ScreenToGif, free) — this is the #1 attention getter
- 'Why this project is interesting' section — mention offline LLM, zero API cost, enterprise patterns
- Complete 'How to Run' with copy-pasteable commands (Docker Compose one-liner)
- Tech stack table with icons (use SimpleIcons or shields.io)
- API documentation link (Swagger UI screenshot)

Resume Bullet Points (copy-paste ready)

- Built an enterprise RAG platform using ASP.NET Core 8, Semantic Kernel, Ollama (Llama 3), and Qdrant, enabling fully offline AI document Q&A with zero API cost
- Designed Clean Architecture solution with CQRS/MediatR, handling document ingestion, chunking, 768-dim vector embedding, and contextual LLM response generation
- Integrated Server-Sent Events for real-time LLM token streaming; built React 18 + TypeScript frontend with JWT auth and markdown chat rendering
- Containerised all 5 services (API, React, Qdrant, PostgreSQL, Ollama) using Docker Compose; set up GitHub Actions CI with xUnit + Testcontainers integration tests

LinkedIn Post Formula

- Hook: 'I built an enterprise AI knowledge assistant — completely offline, zero cloud cost, on my laptop.'
- 3 bullets: what it does, the tech stack, what you learned
- Attach demo GIF directly (not YouTube link — GIFs autoplay in feed)
- Tag: #dotnet #cleanarchitecture #rag #semantickernel #ollama #ai #csharp
- Pin the post to your LinkedIn profile featured section
- Comment with GitHub link in first comment (LinkedIn algo penalises external links in post body)

14. RISK REGISTER & MITIGATIONS

Risk	Likelihood	Impact	Mitigation
Ollama too slow on laptop CPU	High	Medium	Use Mistral 7B (smaller) instead of Llama 3 8B. Enable GPU offload if NVIDIA GPU present. Reduce context window. Document expected inference speed in README.
Qdrant or PostgreSQL Docker issues	Medium	High	Pin Docker image versions in compose file. Use health checks with depends_on. Keep docker-compose.dev.yml for hot-reload without rebuilding images.
nomic-embed-text dimension mismatch	Medium	High	Always verify embedding dimensions (768 for nomic-embed-text) match Qdrant collection config on first run. Add dimension validation in OllamaEmbeddingService.
4-week timeline slipping	Medium	High	Week 1 and 2 are the critical path. React frontend (Week 3) can be simplified to plain HTML/fetch if needed. Prioritise working RAG pipeline over UI polish.
Large PDF causes memory spike	Low	Medium	Process documents asynchronously (background job with IHostedService). Chunk files in streaming batches. Add max file size validation (50MB limit).
JWT secret key management	Low	High	Store JWT secret in environment variable / docker-compose .env file, never hardcode. Add .env to .gitignore. Document in README.

Good luck, Abhishek! This project will stand out in any Senior SDE interview. Ship Week 1 first, stay consistent, and the rest follows. The RAG wave is at its peak — this is exactly what Mastercard, JPMorgan, and SAP interviewers want to see on a resume in 2026.